



Data Visualization

LABORATORIO

Capitolo 9 – Mappe



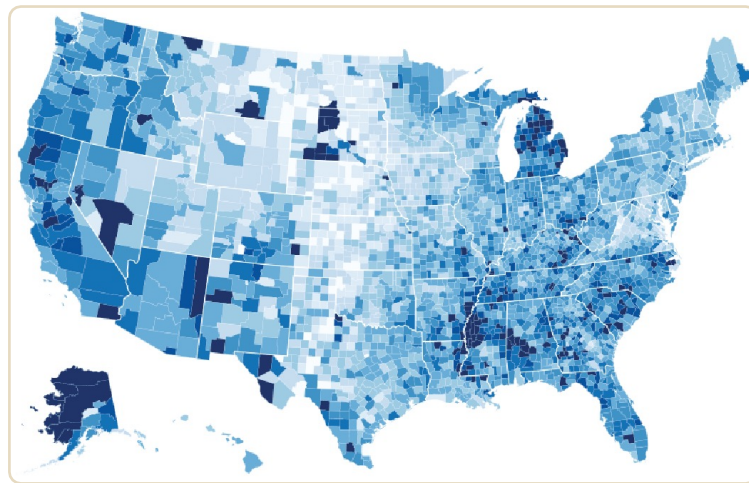
Argomenti del giorno

- **Mappe**
 - GeoJSON
 - Proiezioni
- **Scale Functions**
 - ScaleSequentialSqrt
 - ScaleSqrt

Dati spaziali

I dati **spaziali** (o **geometrici**) rappresentano informazioni legate a posizioni geografiche o coordinate nello spazio.

Sono utili da visualizzare con le **mappe** perché la **posizione è l'informazione chiave** e le mappe permettono di interpretare facilmente relazioni, distribuzioni e pattern spaziali, rendendo i dati più comprensibili e immediati.



Struttura dei dati

Il **GeoJSON** è un formato basato su **JSON** ed è progettato per rappresentare **dati geografici**. Contiene **forme geografiche** come punti, linee o poligoni, e può includere anche informazioni associate a ciascun oggetto come nome di un paese, popolazione, ecc.

Nel nostro caso, abbiamo un oggetto annidato definito come una collezione di *Features*, ogni feature, quindi un paese ha delle proprietà e una *geometria*.

```
{  
  "type": "FeatureCollection",  
  "features": [  
    {  
      "type": "Feature",  
      "properties": {  
        "name": "Afghanistan",  
      },  
      "geometry": {  
        "type": "Polygon",  
        "coordinates": [  
          [[  
            [61.210817, 35.650072],  
            [62.230651, 35.270664],  
            [62.984662, 35.404041],  
            [63.193538, 35.857166],  
          ]],  
        ],  
      },  
    },  
  ],  
}
```

Lettura di un GeoJSON

D3 permette di leggere un file **GeoJSON** con la solita funzione `.json()`. Nel nostro caso abbiamo più file a disposizione.

Carichiamone uno alla volta per vedere come sono strutturati.

In base alla risoluzione del file, lo avremo più preciso e di conseguenza molto più **pesante**.

```
async function map(){ Show usages
  const mapPaths = ['../datasets/africa.geojson',
    '../datasets/africa_hd.geojson',
    '../datasets/usa.geojson',
    '../datasets/world.geojson'];

  getDataJSON(mapPaths[3]) Promise<any>
    .then(data => {
      const idSVG = "mappaSVG";
      renderSimpleMap(data, idSVG);
    }) Promise<void>
    .catch(error => console.error('Error loading data:', error));
```

Proiezione dei dati

Per rappresentare graficamente i dati, d3 mette a disposizione varie funzioni per **proiettare** le coordinate GPS del GeoJSON in coordinate 2D.

Potrebbe essere necessario proiettare i dati in **diversi** modi, per esempio avere una mappa che **mantiene le proporzioni** corrette di ogni stato o una che magari **distorce** le aree ai poli.

La funzione **.geoPath()** permette di generare un path SVG basato sulle coordinate del GeoJSON. Per applicarlo alla proiezione, la si passa tramite **.projection()**

```
const projection =  
  d3.geoAzimuthalEqualArea()  
  // d3.geoMercator()  
  // d3.geoEquirectangular()  
  // d3.geoNaturalEarth1()  
  // d3.geoEqualEarth()  
  // d3.geoConicEqualArea()  
  // d3.geoAlbersUsa()  
  .fitSize([chartWidth, chartHeight], data)  
  
const geoGenerator = d3.geoPath().projection(projection);
```

Costruzione della mappa

Procediamo nel solito modo per creare il grafico, in questo caso come dati utilizziamo le **features**, quindi ogni singolo paese contenuto nel GeoJSON.

Creiamo un **path** per ognuno di essi e definiamo l'attributo d tramite la funzione **geoGenerator** creata prima.

Il risultato **varia** in base al file caricato e alla funzione di **proiezione** utilizzata.

```
gContainer
  .selectAll(null)
  .data(data.features)
  .join('path')
  .attr('d', geoGenerator)
  .attr('fill', 'lightgray')
  .attr('stroke', 'black')
```

Risultati

Proiezione di Mercatore

→ conserva gli angoli/localmente le forme: utile per navigazione e orientamento.

Distorce le aree, soprattutto vicino ai poli

`(d3.geoMercator())`



Proiezione azimutale equivalente di Lambert

→ la sfera viene rappresentata su un disco. Le aree sono conservate bene ma non riproduce fedelmente gli angoli

`(d3.geoAzimuthalEqualArea())`



Proiezione cilindrica equidistante

→ le coordinate geografiche della latitudine e della longitudine come delle coordinate cartesiane. Dunque i poli hanno stessa lunghezza dell'equatore risultando deformati

`(d3.geoEquirectangular())`



Risultati

Proiezione "Natural Earth»

→ è un buon compromesso tra estetica e fedeltà.

Cerca di ridurre distorsioni evidenti

`(d3.geoNaturalEarth1())`



Proiezione "The Equal Earth"

→ mantiene le proporzioni corrette tra le aree.

`(d3.geoEqualEarth())`



Proiezione conica conforme

→ È una proiezione pensata soprattutto per aree a latitudini medie.

Conserva gli angoli, quindi le forme locali sono abbastanza corrette.

`(d3.geoConicConformal())`



Proiezione per gli stati uniti: `(d3.geoAlbersUsa())`

Dataset su più file

Può capitare di non avere tutte le informazioni necessarie a costruire il grafico in un solo file (mappe geografiche tematiche), pertanto è necessario **integrare** con informazioni provenienti da **diversi file**.

`d3.json()` e `d3.csv()` sono funzioni che restituiscono **promise**, oggetti che rappresentano il risultato futuro di un'operazione asincrona.

Per chiamare queste funzioni insieme si utilizza **PromiseAll()** che restituisce il risultato solo dopo che le funzioni all'interno hanno completato il loro ciclo.

```
async function map(){ Show usages
  const mapPaths = ['../datasets/africa.geojson',
    '../datasets/africa_hd.geojson',
    '../datasets/usa.geojson',
    '../datasets/world.geojson'];

  Promise.all(
    [getDataJSON(mapPaths[3]),
     getDataCSV('../datasets/world_population.csv')]
  ).then(([dataJSON, dataCSV]) => {
    const idSVG = "mappaSVG";
    renderMap([dataJSON, dataCSV], idSVG);
  }).catch(error => console.error('Error loading data:', error));
}
```

Bounding box e centroide

Vogliamo fare in modo che al passaggio del mouse sopra uno stato venga mostrato il **centroide** di quello stato e il suo **bounding box**.

Queste informazioni sono utili perché possono esserci stati divisi in aree **non contigue**.

Creiamo degli elementi **rect** e **circle**, recuperiamo il bounding box con la funzione **.bounding()** di **geoGenerator** e il centroide con **.centroid()**, poi aggiorniamo le loro posizioni e dimensioni all'interno di funzioni apposite per gestire gli eventi di passaggio del mouse.

```
function handleMouseOut(e, d, boundsRect, centroidCircle) {  
  boundsRect.attr('width', 0).attr('height', 0);  
  centroidCircle.attr('fill', 'none');  
}
```

renderMap()

```
const hoveredCountryG = gContainer.append('g')  
  
const boundsRect = hoveredCountryG.append('rect')  
  .attr('id', 'bounding-box')  
  .attr('fill', 'none')  
  .attr('stroke', 'black')  
  .attr('stroke-dasharray', '5, 1');  
  
const centroidCircle = hoveredCountryG.append('circle')  
  .attr('id', 'centroid')  
  .attr('r', 3)  
  .attr('fill', 'none');
```

```
gContainer.selectAll('path')  
  .on('mouseover',  
    (e, d) => handleMouseOver(e, d, geoGenerator, boundsRect, centroidCircle))  
  .on('mouseout',  
    (e, d) => handleMouseOut(e, d, boundsRect, centroidCircle))
```

```
function handleMouseOver(e, d, geoGenerator, boundsRect, centroidCircle)  
  const bounds = geoGenerator.bounding(d);  
  boundsRect  
    .attr('x', bounds[0][0])  
    .attr('y', bounds[0][1])  
    .attr('width', bounds[1][0] - bounds[0][0])  
    .attr('height', bounds[1][1] - bounds[0][1]);  
  
  const centroid = geoGenerator.centroid(d);  
  
  centroidCircle  
    .attr('fill', 'red')  
    .attr('cx', centroid[0])  
    .attr('cy', centroid[1]);
```

Legenda dinamica con hovering

geoGenerator() ci permette di ottenere anche altri valori come il **perimetro** e l'**area** degli stati che visualizziamo nella mappa: questi dati sono relativi agli elementi creati da noi, **non sono quelli reali**.

Creiamo quindi una "**legenda**" che mostri le informazioni dello stato puntato. Gestiamo poi il passaggio di queste informazioni nelle funzioni degli eventi create prima.

renderMap()

```
const text_legend = svg.select('#legend-info').select('text');
gContainer.selectAll('path')
  .on('mouseover', (e, d) => {
    handleMouseOver(e, d, geoGenerator, boundsRect, centroidCircle, text_legend);
  })
  .on('mouseout', (e, d) => handleMouseOut(e, d, boundsRect, centroidCircle, text_legend));
```

handleMouseOver()

```
const countryName = d.properties.name;
const pixelArea = geoGenerator.area(d);
const measure = geoGenerator.measure(d);

text_legend.text(`${countryName}
  (Area: ${pixelArea.toFixed(1)} -
  Perimetro: ${measure.toFixed(1)})`);
```

handleMouseOut()

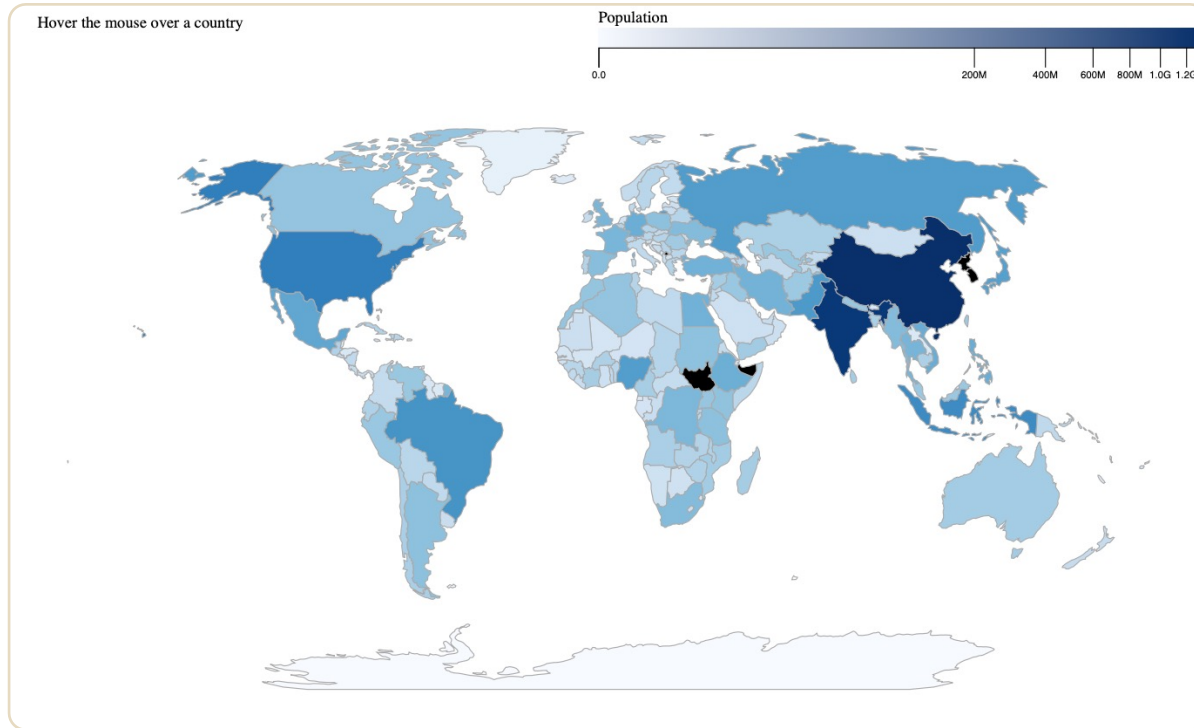
```
function handleMouseOut(e, d, boundsRect, centroidCircle, text_legend) {
  boundsRect.attr('width', 0).attr('height', 0);
  centroidCircle.attr('fill', 'none');
  text_legend.text('Hover over a country');
}
```

Risultato finora

Italy (Area = 334.2 - Perimetro = 174.0)



Obiettivo

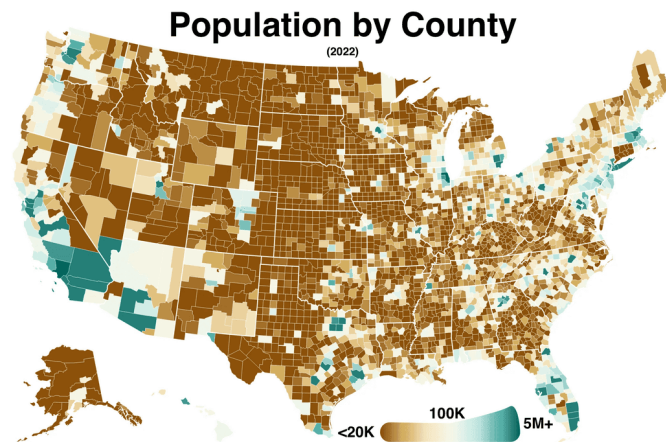


Choropleth map

Una **choropleth map** si distingue da una mappa normale per il modo in cui rappresenta visivamente i dati.

Le aree geografiche vengono colorate, sfumate o tratteggiate in proporzione all'intensità di una specifica variabile statistica.

È lo strumento ideale quando si vuole guardare una mappa e cogliere all'istante tendenze regionali, densità o distribuzioni di un fenomeno.



Gradiente quantitativo

Creiamo la scala colori utilizzando

.scaleSequentialSqrt(), una scala sequenziale non lineare basata sulla radice quadrata, ottima per valori numerici continui e nei casi in cui ci siano **outlier** nel dataset. Perfetta per il nostro caso.

Con **defs** creiamo il **gradiente lineare** e tramite **colorScale** applichiamo lo schema colore. Gli **stop** sono dettati da tutti i colori dello schema.

Creiamo un rettangolo e applicandogli il gradiente e infine aggiungiamo il titolo.

renderMap ()

```
const exp = 0.25;
const colorScale = d3.scaleSequentialSqrt().exponent(exp)
  .domain(d3.extent(population, d => d.pop))
  .interpolator(d3.interpolateBlues);
```

```
function creazioneLegendaColori(svg, margin, svgWidth, legendSize, exp, colorScale) {
  const [legendWidth, legendHeight] = legendSize;

  const defs = svg.append('defs');
  const linearGradient = defs.append('linearGradient')
    .attr('id', 'linear-gradient')
    .attr('x1', '0%').attr('y1', '0%')
    .attr('x2', '100%').attr('y2', '0%');

  linearGradient.selectAll(null)
    .data(colorScale.range())
    .join('stop')
    .attr('offset', (d, i) => i / (colorScale.range().length - 1))
    .attr('stop-color', d => d);

  const legendColor = svg.select('#legend-info').append('g')
    .attr('transform', `translate(${svgWidth-legendWidth-80}, 0)`);

  legendColor.append('text')
    .attr('x', 0)
    .attr('y', -5)
    .text(`Population (exponent: ${exp})`);

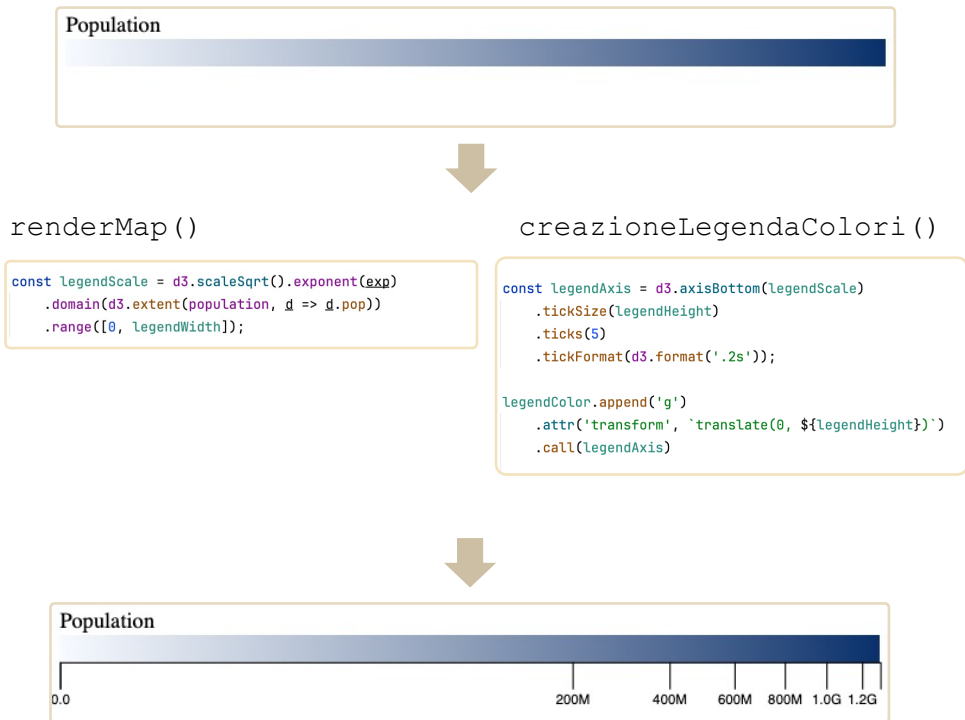
  legendColor.append('rect')
    .attr('width', legendWidth)
    .attr('height', legendHeight)
    .style('fill', 'url(#linear-gradient)');
```


Asse del gradiente

Tramite `.scaleSqrt()`, analoga a `scaleSequentialSqrt()` definiamo una scala basata sulla radice quadrata. Il dominio sarà ancora una volta la **popolazione**, mentre il codominio la grandezza della nostra barra.

Applichiamola quindi alla funzione `.axisBottom()` per costruire l'asse x, definendo il numero di **ticks**.

Vediamo come i ticks non sono posizionati a intervalli regolari, ma a intervalli definiti proprio dalla radice quadrata.



Riempimento mappato

Ora che abbiamo tutto l'occorrente possiamo **mappare** anche il **colore** degli stati in base alla popolazione. Cerchiamo un **match** tra i dati del GeoJSON e i dati del csv, recuperiamo il valore della popolazione e lo mappiamo con la funzione `colorScale`.

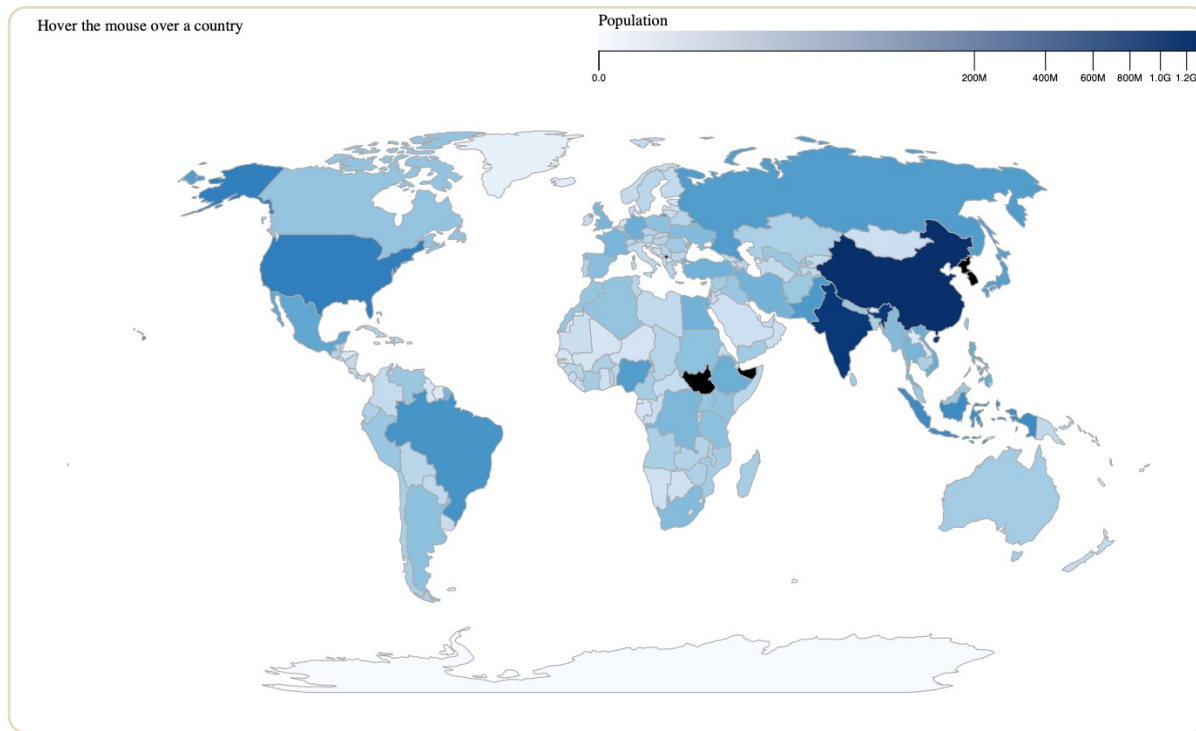
Potrebbe capitare che non ci sia una corrispondenza tra i due file, sia per valori mancanti che errati. Perciò facciamo in modo che quei paesi abbiano un colore di default.

N.B.: è necessario spostare `colorScale` all'inizio di `render_data()`

```
renderMap()
```

```
gContainer
  .selectAll(null)
  .data(dataJSON.features)
  .join('path')
  .attr('d', geoGenerator)
  .attr('stroke', 'black')
  // .attr('fill', 'lightgray')
  .attr('fill', d => {
    const countryData = population
      .find(country => country.code === d.id);
    return countryData ? colorScale(countryData.pop) : 'lightgray';
  })
```

Risultato



Esercizi

1. Con la stessa logica dell'esercizio 6, aggiungere alla legenda la popolazione. Come prima, gestire i casi di informazioni mancanti (es: "Dato sconosciuto").
2. Al passaggio del mouse su uno stato, i bordi diventano rossi e più spessi.

